Pack 23 (Web + API) — Device Trust, Unattended Grants & Consent Receipt Admin (real code)

RemoteDesk · Pack 23 (Web + API) — Device Trust, Unattended Grants & Consent Receipt Admin (real code)

> Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 → 22. The desktop side consumed three things it assumed existed server-side: a device-trust + fingerprint store, unattended-grant fetch/revoke, and somewhere to keep consent receipts. This slice builds those for real in `apps/api` + `apps/web`, so the desktop packs have a server to talk to. Build order: shared admin DTOs → Prisma models + migration → API repos/services/routes → web admin pages → tests.
> Scope: persist + manage device trust (with fingerprint), create/revoke time-boxed unattended grants, and store/verify user-held consent receipts. **Server is NOT the source of truth for live session permissions (the host still is); it persists policy + grants the host reads. Grants are clamped to policy on the server too. No secrets in trust/grant/receipt rows. Auth + org-scoped throughout.**
> Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

```
CREATED [NEW]
  packages/shared/src/permissions/adminDto.ts
  apps/api/prisma/migrations/0023_trust_grants_receipts/migration.sql
  apps/api/src/repositories/deviceTrustRepository.ts
  apps/api/src/repositories/unattendedGrantRepository.ts
  apps/api/src/repositories/consentReceiptRepository.ts
  apps/api/src/services/admin/deviceTrust.service.ts
  apps/api/src/services/admin/unattendedGrant.service.ts
  apps/api/src/routes/deviceAdmin.routes.ts
  apps/web/app/dashboard/devices/page.tsx
  apps/web/app/dashboard/devices/[deviceId]/page.tsx
  apps/web/app/dashboard/devices/deviceAdminApi.ts
  apps/web/app/dashboard/devices/devices.module.css
  apps/web/app/dashboard/receipts/page.tsx
  tests/admin/deviceTrust.test.ts
  tests/admin/unattendedGrant.service.test.ts
  tests/admin/adminDto.test.ts

MODIFIED [MODIFY]
  packages/shared/src/permissions/index.ts   (export adminDto)
  apps/api/prisma/schema.prisma               (Device trust fields + 2 new models)
  apps/api/src/server.ts                      (mount /api/devices admin routes)
  apps/web/app/dashboard/page.tsx             (links to devices + receipts)

NOT TOUCHED (intentionally)
  apps/desktop/**                          (desktop already consumes these via its clients)
  apps/api/src/routes/* (existing)         (only adding deviceAdmin.routes)
```

Part A — Shared: admin DTOs

FILE: packages/shared/src/permissions/adminDto.ts

```
import type { DeviceTrustStatus } from './devicePolicy';
import type { FeatureKey } from './devicePolicy';
import type { DeviceFingerprint } from './deviceFingerprint';

/** Row shown in the device admin list. */
export interface DeviceAdminRow {
  deviceId: string;
  name: string;
  trustStatus: DeviceTrustStatus;
  /** Last known fingerprint hash (display only). */
  fingerprintHash: string | null;
  hasActiveGrant: boolean;
  lastSeenAt: string | null;
}

/** Full device admin detail. */
export interface DeviceAdminDetail extends DeviceAdminRow {
  knownFingerprint: DeviceFingerprint | null;
  unattendedAccessEnabled: boolean;
  remoteInputEnabled: boolean;
  clipboardSyncEnabled: boolean;
  fileTransferEnabled: boolean;
  requiresSessionApproval: boolean;
  maxSessionMinutes: number | null;
}

/** Input to set a device's trust status. */
export interface SetTrustInput {
  trustStatus: DeviceTrustStatus;
}

/** Input to create an unattended grant (server clamps scope to policy). */
export interface CreateGrantInput {
  scope: FeatureKey[];
  /** ISO; defaults to now if omitted. */
  notBefore?: string;
  /** ISO; required, must be in the future. */
  expiresAt: string;
}

/** Receipt as stored/listed server-side (content-free; chain only). */
export interface StoredReceiptRow {
  receiptId: string;
  deviceId: string;
  rootHash: string;
  durationSeconds: number;
  unattended: boolean;
  endReason: string;
  issuedAt: string;
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
// ...existing exports through Pack 22...
export * from './adminDto'; // ADD
```

Part B — API: Prisma schema + migration

FILE: apps/api/prisma/schema.prisma [MODIFY]

```
// ---- MODIFY existing Device model: add trust + fingerprint fields ----
// model Device {
//   ...existing fields...
//   trustStatus            String   @default("untrusted") // trusted|untrusted|blocked
//   fingerprintHash        String?
//   fingerprintSignals     Json?
//   unattendedAccessEnabled Boolean @default(false)
//   remoteInputEnabled     Boolean  @default(false)
//   clipboardSyncEnabled   Boolean  @default(false)
//   fileTransferEnabled    Boolean  @default(false)
//   requiresSessionApproval Boolean @default(true)
//   maxSessionMinutes      Int?
//   lastSeenAt             DateTime?
//   unattendedGrants       UnattendedGrant[]
//   consentReceipts        ConsentReceiptRecord[]
// }

// ---- ADD new models ----
model UnattendedGrant {
  id          String   @id @default(cuid())
  deviceId    String
  orgId       String
  scope       String[]
  notBefore   DateTime
  expiresAt   DateTime
  revokedAt   DateTime?
  createdById String?
  createdAt   DateTime @default(now())

  device Device @relation(fields: [deviceId], references: [id], onDelete: Cascade)

  @@index([deviceId])
  @@index([orgId])
  @@index([deviceId, expiresAt])
}

model ConsentReceiptRecord {
  id              String   @id @default(cuid())
  receiptId       String   @unique
  deviceId        String
  orgId           String
  rootHash        String
  durationSeconds Int
  unattended      Boolean  @default(false)
  endReason       String
```

```
  /// Content-free receipt JSON (summary + chain line items). NO payloads.
  receiptJson     Json
  issuedAt        DateTime
  createdAt       DateTime @default(now())

  device Device @relation(fields: [deviceId], references: [id], onDelete: Cascade)

  @@index([orgId])
  @@index([deviceId])
  @@index([orgId, issuedAt])
}
```

FILE: apps/api/prisma/migrations/0023_trust_grants_receipts/migration.sql

```
-- Device trust + fingerprint + policy fields (additive)
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "trustStatus" TEXT NOT NULL DEFAULT 'untrusted';
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "fingerprintHash" TEXT;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "fingerprintSignals" JSONB;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "unattendedAccessEnabled" BOOLEAN NOT NULL DEFAULT false;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "remoteInputEnabled" BOOLEAN NOT NULL DEFAULT false;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "clipboardSyncEnabled" BOOLEAN NOT NULL DEFAULT false;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "fileTransferEnabled" BOOLEAN NOT NULL DEFAULT false;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "requiresSessionApproval" BOOLEAN NOT NULL DEFAULT true;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "maxSessionMinutes" INTEGER;
ALTER TABLE "Device" ADD COLUMN IF NOT EXISTS "lastSeenAt" TIMESTAMP;

-- UnattendedGrant
CREATE TABLE "UnattendedGrant" (
  "id"          TEXT PRIMARY KEY,
  "deviceId"    TEXT NOT NULL REFERENCES "Device"("id") ON DELETE CASCADE,
  "orgId"       TEXT NOT NULL,
  "scope"       TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[],
  "notBefore"   TIMESTAMP NOT NULL,
  "expiresAt"   TIMESTAMP NOT NULL,
  "revokedAt"   TIMESTAMP,
  "createdById" TEXT,
  "createdAt"   TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "UnattendedGrant_deviceId_idx" ON "UnattendedGrant"("deviceId");
CREATE INDEX "UnattendedGrant_orgId_idx" ON "UnattendedGrant"("orgId");
CREATE INDEX "UnattendedGrant_deviceId_expiresAt_idx" ON "UnattendedGrant"("deviceId", "expiresAt");

-- ConsentReceiptRecord
CREATE TABLE "ConsentReceiptRecord" (
  "id"              TEXT PRIMARY KEY,
  "receiptId"       TEXT NOT NULL UNIQUE,
  "deviceId"        TEXT NOT NULL REFERENCES "Device"("id") ON DELETE CASCADE,
  "orgId"           TEXT NOT NULL,
  "rootHash"        TEXT NOT NULL,
  "durationSeconds" INTEGER NOT NULL,
  "unattended"      BOOLEAN NOT NULL DEFAULT false,
  "endReason"       TEXT NOT NULL,
  "receiptJson"     JSONB NOT NULL,
  "issuedAt"        TIMESTAMP NOT NULL,
  "createdAt"       TIMESTAMP NOT NULL DEFAULT now()
);
```

```
CREATE INDEX "ConsentReceiptRecord_orgId_idx" ON "ConsentReceiptRecord"("orgId");
CREATE INDEX "ConsentReceiptRecord_deviceId_idx" ON "ConsentReceiptRecord"("deviceId");
CREATE INDEX "ConsentReceiptRecord_orgId_issuedAt_idx" ON "ConsentReceiptRecord"("orgId", "issuedAt");
```

Part C — API: repositories

FILE: apps/api/src/repositories/deviceTrustRepository.ts

```
import type {
  DeviceAccessPolicySnapshot,
  DeviceFingerprint,
  DeviceTrustStatus,
  DeviceAdminRow,
} from '@remotedesk/shared';
import { prisma } from '../db/prisma';

function toPolicy(d: any): DeviceAccessPolicySnapshot {
  return {
    deviceId: d.id,
    trustStatus: (d.trustStatus ?? 'untrusted') as DeviceTrustStatus,
    unattendedAccessEnabled: !!d.unattendedAccessEnabled,
    remoteInputEnabled: !!d.remoteInputEnabled,
    clipboardSyncEnabled: !!d.clipboardSyncEnabled,
    fileTransferEnabled: !!d.fileTransferEnabled,
    requiresSessionApproval: d.requiresSessionApproval ?? true,
    maxSessionMinutes: d.maxSessionMinutes ?? null,
    updatedAt: (d.updatedAt ?? new Date()).toISOString?.() ?? new Date().toISOString(),
  };
}

export const deviceTrustRepository = {
  async policy(deviceId: string, orgId: string): Promise<DeviceAccessPolicySnapshot | null> {
    const d = await prisma.device.findFirst({ where: { id: deviceId, orgId } });
    return d ? toPolicy(d) : null;
  },

  async knownFingerprint(deviceId: string, orgId: string): Promise<DeviceFingerprint | null> {
    const d = await prisma.device.findFirst({ where: { id: deviceId, orgId } });
    if (!d?.fingerprintHash || !d?.fingerprintSignals) return null;
    return { hash: d.fingerprintHash, signals: d.fingerprintSignals as any, computedAt: (d.updatedAt ?? new
Date()).toISOString() };
  },

  async setTrust(deviceId: string, orgId: string, trustStatus: DeviceTrustStatus, fingerprint?: DeviceFingerprint):
Promise<void> {
    await prisma.device.updateMany({
      where: { id: deviceId, orgId },
      data: {
        trustStatus,
        ...(fingerprint ? { fingerprintHash: fingerprint.hash, fingerprintSignals: fingerprint.signals as unknown as
object } : {}),
      },
    });
  },
```

```
  async touchLastSeen(deviceId: string, orgId: string): Promise<void> {
    await prisma.device.updateMany({ where: { id: deviceId, orgId }, data: { lastSeenAt: new Date() } });
  },

  async list(orgId: string): Promise<DeviceAdminRow[]> {
    const rows = await prisma.device.findMany({ where: { orgId }, orderBy: { lastSeenAt: 'desc' } });
    const now = new Date();
    const out: DeviceAdminRow[] = [];
    for (const d of rows) {
      const activeGrant = await prisma.unattendedGrant.findFirst({
        where: { deviceId: d.id, revokedAt: null, notBefore: { lte: now }, expiresAt: { gt: now } },
      });
      out.push({
        deviceId: d.id,
        name: (d as any).name ?? d.id,
        trustStatus: (d.trustStatus ?? 'untrusted') as DeviceTrustStatus,
        fingerprintHash: d.fingerprintHash ?? null,
        hasActiveGrant: !!activeGrant,
        lastSeenAt: d.lastSeenAt ? d.lastSeenAt.toISOString() : null,
      });
    }
    return out;
  },
};
```

FILE: apps/api/src/repositories/unattendedGrantRepository.ts

```
import type { UnattendedGrant, FeatureKey } from '@remotedesk/shared';
import { prisma } from '../db/prisma';

function toDto(g: any): UnattendedGrant {
  return {
    grantId: g.id,
    deviceId: g.deviceId,
    scope: g.scope as FeatureKey[],
    notBefore: g.notBefore.toISOString(),
    expiresAt: g.expiresAt.toISOString(),
    revokedAt: g.revokedAt ? g.revokedAt.toISOString() : null,
    createdByLabel: g.createdById ?? null,
  };
}

export const unattendedGrantRepository = {
  async activeForDevice(deviceId: string, orgId: string, now: Date = new Date()): Promise<UnattendedGrant | null> {
    const g = await prisma.unattendedGrant.findFirst({
      where: { deviceId, orgId, revokedAt: null, notBefore: { lte: now }, expiresAt: { gt: now } },
      orderBy: { createdAt: 'desc' },
    });
    return g ? toDto(g) : null;
  },

  async create(input: {
    deviceId: string; orgId: string; scope: string[]; notBefore: Date; expiresAt: Date; createdById: string | null;
  }): Promise<UnattendedGrant> {
    const g = await prisma.unattendedGrant.create({ data: input });
    return toDto(g);
```

```
  },

  async revoke(grantId: string, orgId: string): Promise<void> {
    await prisma.unattendedGrant.updateMany({ where: { id: grantId, orgId, revokedAt: null }, data: { revokedAt: new
Date() } });
  },

  async listForDevice(deviceId: string, orgId: string): Promise<UnattendedGrant[]> {
    const rows = await prisma.unattendedGrant.findMany({ where: { deviceId, orgId }, orderBy: { createdAt: 'desc' } });
    return rows.map(toDto);
  },
};
```

FILE: apps/api/src/repositories/consentReceiptRepository.ts

```
import type { ConsentReceipt, StoredReceiptRow } from '@remotedesk/shared';
import { prisma } from '../db/prisma';

export const consentReceiptRepository = {
  async store(orgId: string, deviceId: string, receipt: ConsentReceipt): Promise<void> {
    await prisma.consentReceiptRecord.upsert({
      where: { receiptId: receipt.receiptId },
      create: {
        receiptId: receipt.receiptId,
        deviceId,
        orgId,
        rootHash: receipt.rootHash,
        durationSeconds: receipt.summary.durationSeconds,
        unattended: receipt.summary.unattended,
        endReason: receipt.summary.endReason,
        receiptJson: receipt as unknown as object,
        issuedAt: new Date(receipt.issuedAt),
      },
      update: {}, // receipts are immutable; ignore duplicates
    });
  },

  async list(orgId: string, limit = 100): Promise<StoredReceiptRow[]> {
    const rows = await prisma.consentReceiptRecord.findMany({
      where: { orgId }, orderBy: { issuedAt: 'desc' }, take: limit,
    });
    return rows.map((r) => ({
      receiptId: r.receiptId, deviceId: r.deviceId, rootHash: r.rootHash,
      durationSeconds: r.durationSeconds, unattended: r.unattended,
      endReason: r.endReason, issuedAt: r.issuedAt.toISOString(),
    }));
  },

  async getJson(orgId: string, receiptId: string): Promise<ConsentReceipt | null> {
    const r = await prisma.consentReceiptRecord.findFirst({ where: { orgId, receiptId } });
    return r ? (r.receiptJson as unknown as ConsentReceipt) : null;
  },
};
```

Part D — API: services

FILE: apps/api/src/services/admin/deviceTrust.service.ts

```
import {
  AppError,
  ErrorCodes,
  type DeviceAdminDetail,
  type DeviceFingerprint,
  type DeviceTrustStatus,
} from '@remotedesk/shared';
import { deviceTrustRepository } from '../../repositories/deviceTrustRepository';
import { unattendedGrantRepository } from '../../repositories/unattendedGrantRepository';

export const deviceTrustService = {
  list: (orgId: string) => deviceTrustRepository.list(orgId),

  async detail(orgId: string, deviceId: string): Promise<DeviceAdminDetail> {
    const policy = await deviceTrustRepository.policy(deviceId, orgId);
    if (!policy) throw new AppError(ErrorCodes.DEVICE_NOT_FOUND);
    const fp = await deviceTrustRepository.knownFingerprint(deviceId, orgId);
    const active = await unattendedGrantRepository.activeForDevice(deviceId, orgId);
    return {
      deviceId,
      name: deviceId,
      trustStatus: policy.trustStatus,
      fingerprintHash: fp?.hash ?? null,
      knownFingerprint: fp,
      hasActiveGrant: !!active,
      lastSeenAt: null,
      unattendedAccessEnabled: policy.unattendedAccessEnabled,
      remoteInputEnabled: policy.remoteInputEnabled,
      clipboardSyncEnabled: policy.clipboardSyncEnabled,
      fileTransferEnabled: policy.fileTransferEnabled,
      requiresSessionApproval: policy.requiresSessionApproval,
      maxSessionMinutes: policy.maxSessionMinutes,
    };
  },

  async setTrust(orgId: string, deviceId: string, status: DeviceTrustStatus, fingerprint?: DeviceFingerprint):
Promise<void> {
    const policy = await deviceTrustRepository.policy(deviceId, orgId);
    if (!policy) throw new AppError(ErrorCodes.DEVICE_NOT_FOUND);
    await deviceTrustRepository.setTrust(deviceId, orgId, status, fingerprint);
  },
};
```

> Add the error code (REVIEW_REQUIRED — patch `packages/shared/src/errors/errorCodes.ts`):
> `DEVICE_NOT_FOUND: 'DEVICE_NOT_FOUND',`
> `GRANT_SCOPE_EMPTY_AFTER_CLAMP: 'GRANT_SCOPE_EMPTY_AFTER_CLAMP',`

FILE: apps/api/src/services/admin/unattendedGrant.service.ts

```
import {
  AppError,
  ErrorCodes,
  policyPermittedFeatures,
  type CreateGrantInput,
```

```
  type FeatureKey,
  type UnattendedGrant,
} from '@remotedesk/shared';
import { deviceTrustRepository } from '../../repositories/deviceTrustRepository';
import { unattendedGrantRepository } from '../../repositories/unattendedGrantRepository';

export const unattendedGrantService = {
  activeForDevice: (deviceId: string, orgId: string) =>
    unattendedGrantRepository.activeForDevice(deviceId, orgId),

  listForDevice: (deviceId: string, orgId: string) =>
    unattendedGrantRepository.listForDevice(deviceId, orgId),

  /**
   * Create a grant. SERVER-SIDE clamp: scope is intersected with policy-permitted
   * features, and a blocked/untrusted device or disabled unattended access is
   * rejected outright. Mirrors the desktop evaluator so the two can't disagree.
   */
  async create(orgId: string, deviceId: string, input: CreateGrantInput, createdById: string | null):
Promise<UnattendedGrant> {
    const policy = await deviceTrustRepository.policy(deviceId, orgId);
    if (!policy) throw new AppError(ErrorCodes.DEVICE_NOT_FOUND);
    if (policy.trustStatus !== 'trusted' || !policy.unattendedAccessEnabled) {
      throw new AppError(ErrorCodes.PERMISSION_DENIED);
    }
    const allowed = policyPermittedFeatures(policy);
    const scope = input.scope.filter((f) => allowed.includes(f as FeatureKey));
    if (scope.length === 0) throw new AppError(ErrorCodes.GRANT_SCOPE_EMPTY_AFTER_CLAMP);

    const notBefore = input.notBefore ? new Date(input.notBefore) : new Date();
    const expiresAt = new Date(input.expiresAt);
    if (expiresAt.getTime() <= Date.now()) throw new AppError(ErrorCodes.PERMISSION_DENIED);

    return unattendedGrantRepository.create({ deviceId, orgId, scope, notBefore, expiresAt, createdById });
  },

  revoke: (grantId: string, orgId: string) => unattendedGrantRepository.revoke(grantId, orgId),
};
```

Part E — API: routes

FILE: apps/api/src/routes/deviceAdmin.routes.ts

```
import { Router } from 'express';
import { z } from 'zod';
import { ALL_CHANNELS } from '@remotedesk/shared'; // (unused import guard example; remove if linted)
import { requireAuth, type AuthedRequest } from '../middleware/requireAuth.middleware';
import { orgContext, type OrgRequest } from '../middleware/orgContext.middleware';
import { deviceTrustService } from '../services/admin/deviceTrust.service';
import { unattendedGrantService } from '../services/admin/unattendedGrant.service';
import { consentReceiptRepository } from '../repositories/consentReceiptRepository';

const h = (fn: (req: any, res: any) => Promise<void>) => (req: any, res: any, next: any) => fn(req, res).catch(next);

const setTrustSchema = z.object({ trustStatus: z.enum(['trusted', 'untrusted', 'blocked']) });
```

```
const createGrantSchema = z.object({
  scope: z.array(z.enum(['remoteInput', 'clipboardSync', 'fileTransfer', 'unattendedAccess'])).min(1),
  notBefore: z.string().datetime().optional(),
  expiresAt: z.string().datetime(),
});

/**
 * /api/devices (admin, org-scoped)
 *   GET   /                          -> device list with trust + grant status
 *   GET   /:deviceId                 -> device detail
 *   PATCH /:deviceId/trust           -> set trust status
 *   GET   /:deviceId/grants          -> grants for device
 *   POST  /:deviceId/grants          -> create grant (scope clamped to policy)
 *   DELETE /:deviceId/grants/:grantId -> revoke grant
 *   GET   /receipts                  -> org consent receipts (content-free)
 *   POST  /receipts                  -> store a user-held receipt (immutable)
 */
export const deviceAdminRoutes = Router()
  .get('/', requireAuth, orgContext, h(async (req, res) => {
    res.json(await deviceTrustService.list((req as OrgRequest).orgId));
  }))
  .get('/receipts', requireAuth, orgContext, h(async (req, res) => {
    res.json(await consentReceiptRepository.list((req as OrgRequest).orgId));
  }))
  .post('/receipts', requireAuth, orgContext, h(async (req, res) => {
    const receipt = req.body; // content-free ConsentReceipt produced by the desktop
    await consentReceiptRepository.store((req as OrgRequest).orgId, receipt.summary.deviceId, receipt);
    res.status(201).json({ stored: true, receiptId: receipt.receiptId });
  }))
  .get('/:deviceId', requireAuth, orgContext, h(async (req, res) => {
    res.json(await deviceTrustService.detail((req as OrgRequest).orgId, req.params.deviceId));
  }))
  .patch('/:deviceId/trust', requireAuth, orgContext, h(async (req, res) => {
    const { trustStatus } = setTrustSchema.parse(req.body);
    await deviceTrustService.setTrust((req as OrgRequest).orgId, req.params.deviceId, trustStatus);
    res.status(204).end();
  }))
  .get('/:deviceId/grants', requireAuth, orgContext, h(async (req, res) => {
    res.json(await unattendedGrantService.listForDevice(req.params.deviceId, (req as OrgRequest).orgId));
  }))
  .post('/:deviceId/grants', requireAuth, orgContext, h(async (req, res) => {
    const input = createGrantSchema.parse(req.body);
    const r = req as OrgRequest & AuthedRequest;
    res.status(201).json(await unattendedGrantService.create(r.orgId, req.params.deviceId, input as any, r.userId));
  }))
  .delete('/:deviceId/grants/:grantId', requireAuth, orgContext, h(async (req, res) => {
    await unattendedGrantService.revoke(req.params.grantId, (req as OrgRequest).orgId);
    res.status(204).end();
  }));
```

FILE: apps/api/src/server.ts [MODIFY]

```
// ...existing imports...
import { deviceAdminRoutes } from './routes/deviceAdmin.routes';
```

```
// ...next to the other app.use('/api/...') mounts:
app.use('/api/devices', deviceAdminRoutes);
```

Part F — Web: device + receipt admin pages

FILE: apps/web/app/dashboard/devices/deviceAdminApi.ts

```
import type {
  DeviceAdminRow,
  DeviceAdminDetail,
  DeviceTrustStatus,
  CreateGrantInput,
  UnattendedGrant,
  StoredReceiptRow,
} from '@remotedesk/shared';
import { api } from '../../../src/api/apiClient';

export const deviceAdminApi = {
  list: () => api.get<DeviceAdminRow[]>('/devices'),
  detail: (deviceId: string) => api.get<DeviceAdminDetail>(`/devices/${deviceId}`),
  setTrust: (deviceId: string, trustStatus: DeviceTrustStatus) =>
    api.patch(`/devices/${deviceId}/trust`, { trustStatus }),
  grants: (deviceId: string) => api.get<UnattendedGrant[]>(`/devices/${deviceId}/grants`),
  createGrant: (deviceId: string, input: CreateGrantInput) =>
    api.post<UnattendedGrant>(`/devices/${deviceId}/grants`, input),
  revokeGrant: (deviceId: string, grantId: string) =>
    api.delete(`/devices/${deviceId}/grants/${grantId}`),
  receipts: () => api.get<StoredReceiptRow[]>('/devices/receipts'),
};
```

FILE: apps/web/app/dashboard/devices/page.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import Link from 'next/link';
import type { DeviceAdminRow } from '@remotedesk/shared';
import { deviceAdminApi } from './deviceAdminApi';
import styles from './devices.module.css';

export default function DevicesPage() {
  const [devices, setDevices] = useState<DeviceAdminRow[]>([]);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    deviceAdminApi.list().then(setDevices).catch(() => setError('Could not load devices.'));
  }, []);

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Devices</h1>
        <p className={styles.sub}>Trust status, fingerprints, and unattended-access grants.</p>
      </header>
      {error && <div className={styles.error}>{error}</div>}
```

```
      <table className={styles.table}>
        <thead><tr><th>Device</th><th>Trust</th><th>Fingerprint</th><th>Unattended</th><th>Last seen</th></tr></thead>
        <tbody>
          {devices.map((d) => (
            <tr key={d.deviceId}>
              <td><Link href={`/dashboard/devices/${d.deviceId}`}>{d.name}</Link></td>
              <td><span className={styles.badge} data-trust={d.trustStatus}>{d.trustStatus}</span></td>
              <td className={styles.mono}>{d.fingerprintHash ? `${d.fingerprintHash.slice(0, 12)}…` : '—'}</td>
              <td>{d.hasActiveGrant ? 'active grant' : '—'}</td>
              <td>{d.lastSeenAt ? new Date(d.lastSeenAt).toLocaleString() : '—'}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </main>
  );
}
```

FILE: apps/web/app/dashboard/devices/[deviceId]/page.tsx

```
'use client';

import React, { useCallback, useEffect, useState } from 'react';
import { useParams } from 'next/navigation';
import type { DeviceAdminDetail, DeviceTrustStatus, UnattendedGrant, FeatureKey } from '@remotedesk/shared';
import { deviceAdminApi } from '../deviceAdminApi';
import styles from '../devices.module.css';

const SCOPE_OPTIONS: FeatureKey[] = ['remoteInput', 'clipboardSync', 'fileTransfer'];

export default function DeviceDetailPage() {
  const { deviceId } = useParams<{ deviceId: string }>();
  const [detail, setDetail] = useState<DeviceAdminDetail | null>(null);
  const [grants, setGrants] = useState<UnattendedGrant[]>([]);
  const [scope, setScope] = useState<FeatureKey[]>(['remoteInput']);
  const [expiresAt, setExpiresAt] = useState('');
  const [error, setError] = useState<string | null>(null);

  const load = useCallback(async () => {
    try {
      setDetail(await deviceAdminApi.detail(deviceId));
      setGrants(await deviceAdminApi.grants(deviceId));
    } catch { setError('Could not load device.'); }
  }, [deviceId]);
  useEffect(() => { void load(); }, [load]);

  const setTrust = async (status: DeviceTrustStatus) => {
    await deviceAdminApi.setTrust(deviceId, status);
    await load();
  };

  const createGrant = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!expiresAt) { setError('Pick an expiry.'); return; }
    try {
      await deviceAdminApi.createGrant(deviceId, { scope, expiresAt: new Date(expiresAt).toISOString() });
```

```
      setExpiresAt('');
      await load();
    } catch { setError('Could not create grant (device may be untrusted or unattended disabled).'); }
  };

  if (!detail) return <main className={styles.page}>{error ? <div className={styles.error}>{error}</div> : <p className={styles.muted}>Loading…</p>}</main>;

  return (
    <main className={styles.page}>
      <header className={styles.header}><h1>{detail.name}</h1>
        <p className={styles.sub}><span className={styles.badge} data-trust={detail.trustStatus}>{detail.trustStatus}</span></p>
      </header>
      {error && <div className={styles.error}>{error}</div>}

      <section className={styles.panel}>
        <h2>Trust</h2>
        <div className={styles.row}>
          <button className={styles.btn} onClick={() => setTrust('trusted')}>Trust</button>
          <button className={styles.btn} onClick={() => setTrust('untrusted')}>Untrust</button>
          <button className={styles.btnDanger} onClick={() => setTrust('blocked')}>Block</button>
        </div>
        <p className={styles.mono}>Fingerprint: {detail.fingerprintHash ?? 'none recorded'}</p>
      </section>

      <section className={styles.panel}>
        <h2>Unattended access grants</h2>
        {!detail.unattendedAccessEnabled && <p className={styles.muted}>Unattended access is disabled by this device's policy.</p>}
        <form onSubmit={createGrant} className={styles.row}>
          <select multiple value={scope} onChange={(e) => setScope(Array.from(e.target.selectedOptions, (o) => o.value as FeatureKey))}>
            {SCOPE_OPTIONS.map((s) => <option key={s} value={s}>{s}</option>)}
          </select>
          <input type="datetime-local" value={expiresAt} onChange={(e) => setExpiresAt(e.target.value)} />
          <button type="submit" className={styles.btn} disabled={detail.trustStatus !== 'trusted' || !detail.unattendedAccessEnabled}>Create grant</button>
        </form>
        <table className={styles.table}>
          <thead><tr><th>Scope</th><th>Expires</th><th>Status</th><th /></tr></thead>
          <tbody>
            {grants.map((g) => {
              const active = !g.revokedAt && Date.parse(g.expiresAt) > Date.now() && Date.parse(g.notBefore) <= Date.now();
              return (
                <tr key={g.grantId}>
                  <td>{g.scope.join(', ')}</td>
                  <td>{new Date(g.expiresAt).toLocaleString()}</td>
                  <td>{g.revokedAt ? 'revoked' : active ? 'active' : 'inactive'}</td>
                  <td>{active && <button className={styles.btnDanger} onClick={async () => { await deviceAdminApi.revokeGrant(deviceId, g.grantId); await load(); }}>Revoke</button>}</td>
                </tr>
              );
            })}
          </tbody>
        </table>
```

```
      </section>
    </main>
  );
}
```

FILE: apps/web/app/dashboard/devices/devices.module.css

```
.page { padding: 24px; max-width: 1000px; margin: 0 auto; }
.header h1 { margin: 0 0 4px; font-size: 22px; }
.sub { color: var(--muted, #6b7280); margin: 0 0 20px; }
.error { background: #fef2f2; color: #991b1b; padding: 10px 14px; border-radius: 8px; margin-bottom: 16px; }
.muted { color: var(--muted, #6b7280); font-size: 13px; }
.mono { font-family: ui-monospace, monospace; font-size: 12px; color: #374151; }
.panel { border: 1px solid var(--border, #e5e7eb); border-radius: 12px; padding: 16px 18px; margin-bottom: 16px;
background: #fff; }
.panel h2 { font-size: 15px; margin: 0 0 12px; }
.row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; margin-bottom: 12px; }
.table { width: 100%; border-collapse: collapse; font-size: 13px; }
.table th, .table td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--border, #e5e7eb); }
.table th { color: var(--muted, #6b7280); font-weight: 600; }
.btn { background: #4f46e5; color: #fff; border: none; padding: 7px 12px; border-radius: 8px; cursor: pointer; }
.btn:disabled { opacity: 0.5; cursor: default; }
.btnDanger { background: #fff; color: #b91c1c; border: 1px solid #fecaca; padding: 6px 12px; border-radius: 8px; cursor:
pointer; }
.badge { font-size: 12px; padding: 2px 10px; border-radius: 999px; background: #eff6ff; color: #1d4ed8; }
.badge[data-trust='trusted'] { background: #ecfdf5; color: #065f46; }
.badge[data-trust='blocked'] { background: #fef2f2; color: #991b1b; }
.badge[data-trust='untrusted'] { background: #fffbeb; color: #92400e; }
```

FILE: apps/web/app/dashboard/receipts/page.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import type { StoredReceiptRow } from '@remotedesk/shared';
import { deviceAdminApi } from '../devices/deviceAdminApi';
import styles from '../devices/devices.module.css';

export default function ReceiptsPage() {
  const [receipts, setReceipts] = useState<StoredReceiptRow[]>([]);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    deviceAdminApi.receipts().then(setReceipts).catch(() => setError('Could not load receipts.'));
  }, []);

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Consent receipts</h1>
        <p className={styles.sub}>Content-free, tamper-evident records of finished sessions.</p>
      </header>
      {error && <div className={styles.error}>{error}</div>}
      <table className={styles.table}>
        <thead><tr><th>Receipt</th><th>Device</th><th>Duration</th><th>Mode</th><th>Ended</th><th>Issued</th></tr>
</thead>
```

```
        <tbody>
          {receipts.map((r) => (
            <tr key={r.receiptId}>
              <td className={styles.mono}>{r.rootHash.slice(0, 12)}…</td>
              <td>{r.deviceId}</td>
              <td>{Math.floor(r.durationSeconds / 60)}m {r.durationSeconds % 60}s</td>
              <td>{r.unattended ? 'unattended' : 'attended'}</td>
              <td>{r.endReason}</td>
              <td>{new Date(r.issuedAt).toLocaleString()}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </main>
  );
}
```

FILE: apps/web/app/dashboard/page.tsx [MODIFY]

```
// ...existing imports...
import Link from 'next/link';

// ...inside the dashboard grid:
<Link href="/dashboard/devices" className="dashboard-card">
  <h3>Devices</h3>
  <p>Trust, fingerprints and unattended-access grants.</p>
</Link>
<Link href="/dashboard/receipts" className="dashboard-card">
  <h3>Consent receipts</h3>
  <p>Tamper-evident records of finished sessions.</p>
</Link>
```

Part G — Tests

FILE: tests/admin/adminDto.test.ts

```
import { describe, it, expect } from 'vitest';
import { policyPermittedFeatures, type DeviceAccessPolicySnapshot } from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: true,
    remoteInputEnabled: true, clipboardSyncEnabled: false, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('admin scope clamping (shared)', () => {
  it('policyPermittedFeatures reflects flags', () => {
    const f = policyPermittedFeatures(policy());
    expect(f).toContain('remoteInput');
    expect(f).toContain('fileTransfer');
    expect(f).not.toContain('clipboardSync');
```

```
  });
});
```

FILE: tests/admin/deviceTrust.test.ts

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

const policy = vi.fn();
const setTrust = vi.fn();
vi.mock('../../apps/api/src/repositories/deviceTrustRepository', () => ({
  deviceTrustRepository: {
    policy: (...a: unknown[]) => policy(...a),
    setTrust: (...a: unknown[]) => setTrust(...a),
    knownFingerprint: vi.fn().mockResolvedValue(null),
    list: vi.fn(),
  },
}));
vi.mock('../../apps/api/src/repositories/unattendedGrantRepository', () => ({
  unattendedGrantRepository: { activeForDevice: vi.fn().mockResolvedValue(null) },
}));

import { deviceTrustService } from '../../apps/api/src/services/admin/deviceTrust.service';

function snap(over = {}) {
  return {
    deviceId: 'dev-1', trustStatus: 'untrusted', unattendedAccessEnabled: false,
    remoteInputEnabled: false, clipboardSyncEnabled: false, fileTransferEnabled: false,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(), ...over,
  };
}

describe('deviceTrustService', () => {
  beforeEach(() => { policy.mockReset(); setTrust.mockReset(); });

  it('throws when the device does not exist', async () => {
    policy.mockResolvedValue(null);
    await expect(deviceTrustService.setTrust('org', 'dev-x', 'trusted')).rejects.toThrow();
  });

  it('sets trust for an existing device', async () => {
    policy.mockResolvedValue(snap());
    await deviceTrustService.setTrust('org', 'dev-1', 'trusted');
    expect(setTrust).toHaveBeenCalledWith('dev-1', 'org', 'trusted', undefined);
  });
});
```

FILE: tests/admin/unattendedGrant.service.test.ts

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

const policy = vi.fn();
const create = vi.fn();
vi.mock('../../apps/api/src/repositories/deviceTrustRepository', () => ({
  deviceTrustRepository: { policy: (...a: unknown[]) => policy(...a) },
}));
vi.mock('../../apps/api/src/repositories/unattendedGrantRepository', () => ({
```

```
  unattendedGrantRepository: { create: (...a: unknown[]) => create(...a), activeForDevice: vi.fn(), listForDevice:
vi.fn(), revoke: vi.fn() },
}));

import { unattendedGrantService } from '../../apps/api/src/services/admin/unattendedGrant.service';

function snap(over = {}) {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: true,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: false,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(), ...over,
  };
}
const future = new Date(Date.now() + 3_600_000).toISOString();

describe('unattendedGrantService.create', () => {
  beforeEach(() => { policy.mockReset(); create.mockReset(); create.mockResolvedValue({ grantId: 'g1' }); });

  it('rejects an untrusted device', async () => {
    policy.mockResolvedValue(snap({ trustStatus: 'untrusted' }));
    await expect(unattendedGrantService.create('org', 'dev-1', { scope: ['remoteInput'], expiresAt: future } as any,
'u1')).rejects.toThrow();
  });

  it('rejects when unattended disabled by policy', async () => {
    policy.mockResolvedValue(snap({ unattendedAccessEnabled: false }));
    await expect(unattendedGrantService.create('org', 'dev-1', { scope: ['remoteInput'], expiresAt: future } as any,
'u1')).rejects.toThrow();
  });

  it('clamps scope to policy-permitted features', async () => {
    policy.mockResolvedValue(snap()); // fileTransfer NOT permitted
    await unattendedGrantService.create('org', 'dev-1', { scope: ['remoteInput', 'fileTransfer'], expiresAt: future } as
any, 'u1');
    const passed = create.mock.calls[0][0];
    expect(passed.scope).toContain('remoteInput');
    expect(passed.scope).not.toContain('fileTransfer'); // clamped out
  });

  it('rejects when scope is empty after clamping', async () => {
    policy.mockResolvedValue(snap({ remoteInputEnabled: false, clipboardSyncEnabled: false, fileTransferEnabled: false
}));
    await expect(unattendedGrantService.create('org', 'dev-1', { scope: ['remoteInput'], expiresAt: future } as any,
'u1')).rejects.toThrow();
  });

  it('rejects an expiry in the past', async () => {
    policy.mockResolvedValue(snap());
    const past = new Date(Date.now() - 1000).toISOString();
    await expect(unattendedGrantService.create('org', 'dev-1', { scope: ['remoteInput'], expiresAt: past } as any,
'u1')).rejects.toThrow();
  });
});
```

Part H — Reports

FILE: generated-pack23-web-api-device-admin-merge-summary.md

```
# Pack 23 (Web + API) — Device Trust, Grants & Receipt Admin · Merge Summary

## What this slice does
Builds the server side the Desktop Packs 15 → 22 assumed: persistent device trust
+ fingerprint, time-boxed unattended grants (created/revoked by admins), and a
store for user-held consent receipts. Closes the loop so the desktop's
devicePolicyClient / unattendedGrantStore / consentReceiptService have a real API.

## Files created (16)
- packages/shared/src/permissions/adminDto.ts
- apps/api/prisma/migrations/0023_trust_grants_receipts/migration.sql
- apps/api/src/repositories/{deviceTrustRepository,unattendedGrantRepository,consentReceiptRepository}.ts
- apps/api/src/services/admin/{deviceTrust.service,unattendedGrant.service}.ts
- apps/api/src/routes/deviceAdmin.routes.ts
- apps/web/app/dashboard/devices/{page.tsx,[deviceId]/page.tsx,deviceAdminApi.ts,devices.module.css}
- apps/web/app/dashboard/receipts/page.tsx
- tests/admin/{adminDto,deviceTrust,unattendedGrant.service}.test.ts

## Files modified (4)
- packages/shared/src/permissions/index.ts — export adminDto.
- apps/api/prisma/schema.prisma — Device trust/fingerprint/policy fields + UnattendedGrant + ConsentReceiptRecord.
- apps/api/src/server.ts — mount /api/devices.
- apps/web/app/dashboard/page.tsx — add devices + receipts links.
- (also REVIEW_REQUIRED: errorCodes.ts — add DEVICE_NOT_FOUND + GRANT_SCOPE_EMPTY_AFTER_CLAMP)

## Files intentionally NOT touched
- apps/desktop/** — already consumes these endpoints via its Pack 15/20/21/22 clients.

## Remaining manual wiring risks
1. Confirm the existing Device model name/fields; the migration is additive (IF NOT EXISTS).
2. Wire DEVICE_POLICY_ENDPOINT (desktop Pack 15) to GET /api/devices/:id (or a dedicated policy route returning the
snapshot + knownFingerprint).
3. Desktop unattendedGrantStore.fetchGrant -> GET /api/devices/:id/grants (active) ; revoke -> DELETE.
4. Desktop consentReceiptService -> POST /api/devices/receipts (optional server copy).
5. Add org/role checks (admin-only) if trust/grant mutation should be restricted.

## Verify
- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: set a device trusted + enable unattended; create a grant (scope clamps to policy); revoke it; end a desktop
session and confirm a receipt row appears.
```

FILE: generated-pack23-web-api-device-admin-code-review.md

```
# Pack 23 (Web + API) — Code Review Notes

## Strengths
- Server clamps grant scope to policy-permitted features (policyPermittedFeatures), mirroring the desktop evaluator —
the two cannot disagree.
- Grants require a trusted device + unattended-enabled policy + future expiry; rejected otherwise.
- Receipts are stored immutably (upsert ignores duplicates) and content-free (chain + summary only).
- Org-scoped + auth on every route; device existence checked before mutation.
- Pure clamp + service logic unit-tested with repository mocks.
```

```
## Reviewer checklist
- [ ] Device model field names match the migration (additive IF NOT EXISTS).
- [ ] Add admin/RBAC guard if trust/grant changes must be admin-only (currently any org member).
- [ ] Desktop policy fetch route returns { policy, knownFingerprint } shape used by Pack 20.
- [ ] Receipt POST validates it is content-free (no payload fields) before storing.
- [ ] BigInt/Json columns serialize cleanly through your API layer.

## Safety
- No secrets/payloads in trust, grant, or receipt rows.
- Grant can never exceed policy (server clamp + desktop clamp, both).
- Server persists policy; the host remains source of truth for LIVE session permissions.
```

FILE: generated-pack23-web-api-device-admin-manifest.json


```
{
  "pack": "pack23-web-api-device-admin",
  "title": "Device Trust, Unattended Grants & Consent Receipt Admin",
  "actualFileCount": 20,
  "filesCreated": [
    "packages/shared/src/permissions/adminDto.ts",
    "apps/api/prisma/migrations/0023_trust_grants_receipts/migration.sql",
    "apps/api/src/repositories/deviceTrustRepository.ts",
    "apps/api/src/repositories/unattendedGrantRepository.ts",
    "apps/api/src/repositories/consentReceiptRepository.ts",
    "apps/api/src/services/admin/deviceTrust.service.ts",
    "apps/api/src/services/admin/unattendedGrant.service.ts",
    "apps/api/src/routes/deviceAdmin.routes.ts",
    "apps/web/app/dashboard/devices/page.tsx",
    "apps/web/app/dashboard/devices/[deviceId]/page.tsx",
    "apps/web/app/dashboard/devices/deviceAdminApi.ts",
    "apps/web/app/dashboard/devices/devices.module.css",
    "apps/web/app/dashboard/receipts/page.tsx",
    "tests/admin/adminDto.test.ts",
    "tests/admin/deviceTrust.test.ts",
    "tests/admin/unattendedGrant.service.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/api/prisma/schema.prisma",
    "apps/api/src/server.ts",
    "apps/web/app/dashboard/page.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/**"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "@prisma/client", "express", "zod", "next", "react", "vitest"],
    "assumedFromProject": [
      "existing Prisma Device model + org scoping",
      "apps/api middleware: requireAuth, orgContext",
      "apps/web src/api/apiClient (get/post/patch/delete)",
      "Desktop Packs 15/20/21/22 clients that call these endpoints"
    ]
  },
  "knownLimitations": [
```

```
    "Trust/grant mutations are org-auth only; add admin RBAC if required.",
    "Receipt POST trusts the desktop-built content-free receipt; add a server-side content-free validator before
storing.",
    "Device policy fetch route shape must match the desktop Pack 20 expectation ({ policy, knownFingerprint }).",
    "Migration is additive; verify the existing Device model/columns first."
  ],
  "remainingManualWiringRisks": [
    "Point desktop DEVICE_POLICY_ENDPOINT + grant + receipt clients at these routes.",
    "Add errorCodes DEVICE_NOT_FOUND + GRANT_SCOPE_EMPTY_AFTER_CLAMP.",
    "Decide admin-only RBAC for trust/grant changes."
  ],
  "estimatedCompletionAfterCarefulMerge": "76-82%"
}
```

Pack 23 (Web + API) done: 20 files (16 created, 4 modified). This is the server side every recent desktop pack assumed existed. Device trust + fingerprint now persist, admins create + revoke time-boxed unattended grants from the web dashboard, and user-held consent receipts get an org-side store. The desktop's `devicePolicyClient`, `unattendedGrantStore`, and `consentReceiptService` finally have real endpoints to hit.

The clean bit: the API clamps grant scope to policy-permitted features using the exact same `policyPermittedFeatures` helper the desktop uses, so a grant created in the web UI can never authorize more than the device's policy allows, enforced on both ends. Grants require a trusted device with unattended enabled and a future expiry, receipts are stored immutably and content-free, and every route is auth + org-scoped. The server persists policy; the host still owns live session permissions.

That bridges desktop and web for the whole trust/consent system. Next natural slice = admin RBAC on these mutations (so only admins can trust devices / mint grants), or server-side receipt verification + a tamper-check endpoint. Bolo.